

Arbitrary code execution in LLMMathChain #8363

✓ Closed



jan-kubena opened on Jul 28, 2023



System Info

Langchain version: 0.0.244

Numexpr version: 2.8.4

Python version: 3.10.11

Who can help?

[@hwchase17](#) [@vowelparrot](#)

Information

- ☐ The official example notebooks/scripts
- ☐ My own modified scripts

Related Components

- ☐ LLMs/Chat Models
- ☐ Embedding Models
- ☐ Prompts / Prompt Templates / Prompt Selectors
- ☐ Output Parsers
- ☐ Document Loaders
- ☐ Vector Stores / Retrievers
- ☐ Memory
- ☐ Agents / Agent Executors
- ☐ Tools / Toolkits
- ☒ Chains
- ☐ Callbacks/Tracing
- ☐ Async

Reproduction

Numexpr's evaluate function that Langchain uses [here](#) in the LLMMathChain is susceptible to arbitrary code execution with eval in the latest released version. See this [issue](#) where PoC for numexpr's evaluate is also provided.

This vulnerability allows an arbitrary code execution, that is to run code and commands on target machine, via LLMMathChain's run method with the right prompt. I'd like to ask the Langchain's maintainers to confirm if they want a full PoC with Langchain posted here publicly.

Expected behavior

Numerical expressions should be evaluated securely so as to not allow code execution.

2



 dosubot added **bug** on Jul 28, 2023

 hinthornw on Jul 28, 2023

Collaborator ...

Thanks for raising! We are working to move the LLMMathChain out of the langchain core library, for reasons such as this

 dosubot **bot** on Jul 28, 2023

...

Answer generated by a

Answer

I understand your concern about the potential security vulnerability in the LLMMathChain class of LangChain. You've correctly identified that the `numexpr.evaluate` function, which is used in the `_evaluate_expression` method, could potentially execute arbitrary code if a malicious expression is passed to it. This is due to the fact that `numexpr.evaluate` uses Python's `eval` function, which can execute arbitrary Python code.

Here's the code snippet in question:

```
def _evaluate_expression(self, expression: str) -> str:
    try:
        local_dict = {"pi": math.pi, "e": math.e}
        output = str(
            numexpr.evaluate(
                expression.strip(),
                global_dict={}, # restrict access to globals
                local_dict=local_dict, # add common mathematical functions
            )
        )
    except Exception as e:
        raise ValueError(
            f'LLMMathChain._evaluate("{expression}") raised error: {e}.'
            " Please try again with a valid numerical expression"
        )
```



```
# Remove any leading and trailing brackets from the output
return re.sub(r"^\[|\]$", "", output)
```

In this code, `numexpr.evaluate` is called with the `expression` argument, which could potentially contain malicious code. Although the `global_dict` argument is set to an empty dictionary to restrict access to global variables, and the `local_dict` argument is set to a dictionary containing common mathematical functions, this does not prevent the execution of arbitrary code.

This means that a malicious user could potentially execute arbitrary code on the server where LangChain is running by passing a specially crafted expression to the `LLMMathChain`'s `run` method. This is indeed a serious security issue that needs to be addressed.

However, please note that this is just one potential interpretation of the issue. If you believe this interpretation is correct and that this is a bug that could impact other users, I encourage you to make a pull request to address this issue. If your experience contradicts this interpretation, I would suggest upgrading to the latest version of LangChain to see if the issue persists.

Sources

- [libs/langchain/langchain/chains/llm_math/base.py](https://github.com/langchain-ai/langchain/blob/master/langchain/chains/llm_math/base.py)

This response is meant to be useful, save you time, and share context. It is **not meant to be a precise solution**, but rather a starting point for your own research.

Help me be more useful! Please leave a 👍 if this is helpful and 👎 if it is irrelevant.

4

1

zoegevens on Aug 1, 2023 · edited by zoegevens

Edits ▾

Contributor

...

@hinthornw As an alternative, we could use `sympy.simplify` ? It's a common alternative to `eval` / `numexpr` (see this thread on vulnerabilities in [numexpr](#)). As well as avoiding that vulnerability, `sympy` also supports a much larger range of mathematical operations (i.e. rounding, factorials, etc.) and is about 10x as common as `numexpr` (as measured by github stars), so I'd guess its easier for an LLM to generate syntactically correct input (in fact, I spent about 10mins trying to get GPT-3 to generate a mathematical input that `sympy` *couldn't* process, and failed).

I was testing some multi-step QA workflows inspired by a Langchain multi-step agent QA [workflow](#), and was using the `llm-math` tool in an Agent chain, with prompts like: "Who is Zoe Saldana's partner? What is their current age raised to the 0.43 power, rounded to 3sf?". This would cause `numexpr` to throw an error, since the chain would try and fail to pass it some kind of rounding instruction. This also happens with the factorial function in [#3071](#). Both of these use cases are fixed by backing `LLMMathchain` with a `simplify` call.

Happy to attach some examples / create a PR to scope out the replacement if people agree. It's what I've been using locally and it works great!

hinthornw on Aug 1, 2023

Collaborator

...

That makes sense to me. I'd be happy to review a PR! Thank you for being proactive about this!

jan-kubena on Aug 1, 2023 · edited by jan-kubena

Edits ▾

Author



I'd just like to point out that there still needs to be some kind of validation/protection because sympy.simplify uses eval and shouldn't be used on unsanitized input as per official [docs](#). I think it could also be worth looking at [this](#) langchain PR that implemented sympy in RestrictedPython as an inspiration.

zoeqevans on Aug 1, 2023

Contributor



Ah damn, yeah I missed that: was foolishly going off the StackOverflow thread and a memory of a still-unresolved effort within [Sympify](#) to remove eval / add a safe-mode.

`sympify` still offers a lot more functionality than `numexpr`, so I'd still be in favour of swapping anyway, but agree that there aren't any extra security benefits vs. `numexpr` on the eval front.



zoeqevans mentioned this [on Aug 2, 2023](#)

[Swap in Sympy instead of numexpr for more expressive LLMMath tool #8627](#)

zoeqevans on Aug 2, 2023

Contributor



[@hinthornw](#) [PR here](#)

♥ 3

jan-kubena on Sep 13, 2023

Author



Hi all, input sanitization has been added in numexpr version 2.8.6 for `evaluate` by default (see this [issue](#) I mentioned previously). As far as I can tell, it looks to be pretty secure. It'd be nice for the expression execution to be done in a secure container/environment, but I think that for now, the sanitization that numexpr does is surely better than nothing.

Since that is the case, I'd like to ask [@hinthornw](#) to advise on the preferable next course of action, that is whether to just bump the numexpr version, replace numexpr completely with Sympy and a secure container/environment (since it'd be vulnerable by itself) or other solutions to this. Thank you!

tabdunabi on Sep 27, 2023



Hi [@hinthornw](#), Any ETA on resolving this issue?
Thanks

elmegatan26 on Sep 28, 2023



This vulnerability is getting flagged by InfoSec teams. Any idea on when the update is being released?

eyurtsev on Oct 3, 2023 · edited by eyurtsev

Edits ▾

Collaborator



[@elmegatan26](#) , [@tabdunabi](#) , [@jan-kubena](#) , numexpr is now (on master) an optional dependency, we've also added a constraint to specify that code only works with `>=2.8.6` (which has input sanitization).

- Is this enough to address concerns flagged by InfoSec teams?
- Also do you mind sharing if you're using LLMathChain yourself and if so which mathematical operations do you rely on?



[eyurtsev](#) self-assigned this [on Oct 3, 2023](#)

tabdunabi on Oct 3, 2023



[@eyurtsev](#), for our use case, we do not use `LLMathChain` , but our security scans detect the `numexpr` vulnerability. This is a critical issue for us and blocker to publish our solutions, used by our customers. If making `numexpr` optional (not installed by default with `LangChain`), then this is enough for us.

10 remaining items

Load more

eyurtsev on Oct 11, 2023

Collaborator



Hi [@tabdunabi](#), realistic timeline is 1-3 weeks. Are you relying on any of the agents; i.e., the pandas agent, xorbis agent or spark agent (which dependent on the python ast tool?).

dvirginz on Oct 11, 2023



If we aren't using agents or LLMMathChain, is there a clear version we can already use?

tabdunabi on Oct 12, 2023



Thank you [@eyurtsev](#) for the update.

Currently, we are not using LangChain agents. However, our build pipeline fails because of any security vulnerabilities in the libraries shipped with our solutions. So, even though we are not using LangChain agents, we need to go through a formal approval process with our security team, and prove the vulnerable code is not actually used by our solutions, to be able to get an exception to release. Additionally, once we publish our code on GitHub, GitHub Dependabot will flag these security vulnerabilities, and we need to address auto-cut tickets for our team.

So, it would be much easier, and safer, to ship code with zero security vulnerabilities in downstream dependencies.

eyurtsev on Oct 12, 2023

Collaborator ...

Targeting end of month 10/28 (will announce in a bit with expected changes) to resolve the CVE to allow existing users to migrate. In the meantime, are you able to fork and remove affected code?

tabdunabi on Oct 12, 2023

...

Thank you [@eyurtsev](#)!.
Creating a fork is not an option for us. We want our customers to be able to build our code by themselves using published libraries, available on Pypi/NPM. We do not want to maintain forks of external libraries.

eyurtsev on Oct 12, 2023

Collaborator ...

<https://github.com/langchain-ai/langchain/discussions/11680> -- announcement

1

eyurtsev on Oct 28, 2023 · edited by eyurtsev

Edits ▾ Collaborator ...

Python AST tool CVE was resolved here: [#12427](#) cc [@tabdunabi](#) / [@dvirginz](#)
(The original CVE for LLMathChain was resolved a while back -- closing this issue.)
As of release: <https://github.com/langchain-ai/langchain/releases/tag/v0.0.325>

1

 [eyurtsev](#) closed this as completed on Oct 28, 2023

tabdunabi on Oct 28, 2023

...

Thank you [@eyurtsev](#) !. We will immediately upgrade LangChain version, used by our code, to `v0.0.325`

dvirginz on Oct 28, 2023 via email · edited by dvirginz

Edits ▾ ...

Thank you. It seems that Snyk still identifies 1 high-risk CVE in LangChain. Any thoughts? Thanks.

<https://security.snyk.io/package/pip/langchain>

eyurtsev on Oct 28, 2023

Collaborator ...

[@dvirginz](#) that's the CVE that got patched with this release. It can take up to several days for the CVE to information to be updated in the relevant databases.

mschirmer84 on Nov 15, 2023

...

Can this [\[CVE-2023-39631\]](#) (<http://web.nvd.nist.gov/view/vuln/detail?vulnId=CVE-2023-39631>) be closed off with the above fix?

eyurtsev on Feb 13, 2024

Collaborator ...

That CVE was resolved as well a while back: [GHSA-f73w-4m7g-ch9x](#)

Locking conversation



langchain-ai locked as resolved and limited conversation to collaborators [on Feb 13, 2024](#)



eyurtsev added **security** [on Mar 14, 2024](#)

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees



eyurtsev

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Participants



+3